UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
Departamento de Computação - DC
Graduação em Ciência da Computação
Disciplina: Sistemas Distribuídos
Prof.: Paulo Henrique Lopes Silva

Prática 2.5 - Sincronizar acesso a coleções

1. Crie uma classe de aplicação com uma estrutura de dados (coleção) para simular um recurso compartilhado.

Classe App.java

```java
public class App {

	static List<String> lista = new ArrayList<>();

	public static void main(String[] args) {


	}

}
```

2. Crie uma classe para implementar uma thread que acessa este recurso.

Classe ThreadAR.java

```java
public class ThreadAR implements Runnable{

	@Override
	public void run() {

		String nome = Thread.currentThread().getName();
		App.lista.add(nome);
		System.out.println(nome + " inseriu na lista!");

	}

}
```

3. Na aplicação:

Classe App.java

```java
public class App {

	static List<String> lista = new ArrayList<>();
```

```
    public static void main(String[] args) throws InterruptedException {

        //lista = Collections.synchronizedList(lista);

        ThreadAR tac = new ThreadAR();

        Thread t0 = new Thread(tac);
        Thread t1 = new Thread(tac);
        Thread t2 = new Thread(tac);

        t0.start();
        t1.start();
        t2.start();

        Thread.sleep(300);

        System.out.println(lista);
    }

}
```

- **Condições de corrida.**
  - Cenário Problemático:
    - Considere que a seguinte sequência de eventos sem sincronização:
      - A Thread A e a Thread B acessam a lista ao mesmo tempo.
      - Ambas verificam o tamanho da lista e tentam adicionar um novo elemento.
      - Antes que a Thread A conclua sua escrita, a Thread B também modifica a estrutura da lista.
      - Isso pode resultar em elementos sendo sobrescritos, perdidos ou até mesmo uma exceção de concorrência *(ConcurrentModificationException)*.

- **Problemas com visibilidade e reordenação de instruções.**
  - O que acontece?
    - Como a lista não é sincronizada, a memória compartilhada entre threads pode não estar atualizada corretamente.
    - Uma thread pode estar escrevendo na lista, mas outra thread pode não ver essa alteração imediatamente devido à caching da CPU ou reordenação de instruções pelo compilador.
  - Exemplo prático.
    - A Thread A adiciona um elemento à lista, mas essa alteração ainda não foi propagada para a memória principal.
    - A Thread B lê a lista antes que a atualização da Thread A tenha sido visível para ela.
    - Como resultado, a Thread B pode acabar sobrescrevendo um valor sem perceber.

- **Problemas com o crescimento da ArrayList.**
  - O que acontece?
    - O ArrayList cresce dinamicamente conforme elementos são adicionados.
    - Quando o limite interno da capacidade do ArrayList é atingido, ele cria um novo array interno maior e copia os elementos antigos para o novo.

        - Se duas threads tentarem adicionar ao mesmo tempo, uma delas pode estar trabalhando com uma versão antiga do array, resultando em elementos perdidos.
    - Cenário Problemático:
        - A Thread A está no meio de uma operação de expansão da ArrayList.
        - A Thread B tenta adicionar um item ao mesmo tempo.
        - Como a expansão não é atômica, a Thread B pode acabar trabalhando com um array desatualizado, levando a perda de dados.

4. Como garantir que todas as threads escrevam na lista?

1. Usar *Collections.synchronizedList*.

```
Java
static List<String> lista =
                    Collections.synchronizedList(new ArrayList<>());
```

- Garante que todas as operações de leitura e escrita sejam feitas de forma segura.

2. Sincronizar manualmente com *synchronized*.

```
Java
synchronized (lista) {
       lista.add(Thread.currentThread().getName() + "-Item");
}
```

- Evita condições de corrida, garantindo que apenas uma thread por vez acesse a lista.

3. Usar *CopyOnWriteArrayList* (alternativa mais segura).

```
Java
static List<String> lista = new CopyOnWriteArrayList<>();
```

- Essa lista cria uma cópia da estrutura ao invés de modificar diretamente o array interno, evitando problemas de concorrência.